Homework 1

Group 9: Daniel Borges – 109479, Duarte Cruz – 110181

Ex1- a)

1- a) $X_1 = (-3, -2, -1, 0, 1, 2)$ $\quad X_2 = f(X_1) = 0.5X_1$

$X_2 = (-1.5, -1, -0.5, 0, 0.5, 1)$

$\bar{X}_1 = \frac{-3 \cancel{-2} \cancel{-1} \cancel{+0} \cancel{+1} \cancel{+2}}{6} = \frac{-3}{6} = -0.5$

$\bar{X}_2 = \frac{-1.5 \cancel{-1} \cancel{-0.5} \cancel{+0} \cancel{+0.5} \cancel{+1}}{6} = -\frac{1.5}{6} = -0.25$

x_{1i}	$x_{1i} - \bar{x}_1$	$(x_{1i} - \bar{x}_1)^2$
-3	-2.5	6.25
-2	-1.5	2.25
-1	-0.5	0.25
0	0.5	0.25
1	1.5	2.25
2	2.5	6.25

$\rightarrow \sum_{i=1}^{n} (x_{1i} - \bar{x}_1)^2 = 17.5 \rightarrow s_{x_1} = \sqrt{\frac{17.5}{5}} = \frac{\sqrt{14}}{2}$

x_{2i}	$x_{2i} - \bar{x}_2$	$(x_{2i} - \bar{x}_2)^2$
-1.5	-1.25	1.5625
-1	-0.75	0.5625
-0.5	-0.25	0.0625
0	0.25	0.0625
0.5	0.75	0.5625
1	1.25	1.5625

$\rightarrow \sum_{i=1}^{n} (x_{2i} - \bar{x}_2)^2 = 4.375 \rightarrow s_{x_2} = \sqrt{\frac{4.375}{5}} = \frac{\sqrt{14}}{4}$

$(x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)$
3.125
1.125
0.125
0.125
1.125
3.125

$\rightarrow \sum_{i=1}^{n} (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2) = 8.75 \rightarrow Cov(x_1, x_2) = \frac{8.75}{5} = 1.75$

$\boxed{\text{Pearson} \rightarrow \frac{1.75}{\frac{\sqrt{14}}{2} \times \frac{\sqrt{14}}{4}} = 1}$ → perfect positive linear correlation

Rank X_1	X_1	X_2	Rank X_2
1	-3	-1.5	1
2	-2	-1	2
3	-1	-0.5	3
4	0	0	4
5	1	0.5	5
6	2	1	6

Since Rank X_1 = Rank X_2, $\boxed{\text{Spearman} = 1}$ → perfect positive monotonic correlation

Ex1-b)

b) $X_1 = (-3, -2, -1, 0, 1, 2)$ $\quad X_2 = f(X_1) = \max(0, X_1)$

$X_2 = (0, 0, 0, 0, 1, 2)$ $\quad \bar{X}_2 = \frac{1+2}{6} = \frac{3}{6} = 0.5$

$S_{X_1} = \frac{\sqrt{14}}{2}$ ← calculated in 1a)

X_{2i}	$X_{2i} - \bar{X}_2$	$(X_{2i} - \bar{X}_2)^2$
0	-0.5	0.25
0	-0.5	0.25
0	-0.5	0.25
0	-0.5	0.25
1	0.5	0.25
2	1.5	2.25

$\rightarrow \sum_{i=1}^{n} (X_{2i} - \bar{X}_2)^2 = 3.5 \rightarrow S_{X_2} = \sqrt{\frac{3.5}{5}} = \frac{\sqrt{70}}{10}$

calculated in 1a)

$X_{1i} - \bar{X}_1$	$X_{2i} - \bar{X}_2$	$(X_{1i} - \bar{X}_1)(X_{2i} - \bar{X}_2)$
-2.5	-0.5	1.25
-1.5	-0.5	0.75
-0.5	-0.5	0.25
0.5	-0.5	-0.25
1.5	0.5	0.75
2.5	1.5	0.375

$\rightarrow \sum_{i=1}^{n} (X_{1i} - \bar{X}_1)(X_{2i} - \bar{X}_2) = 6.5$

$Cov(X_1, X_2) = \frac{6.5}{5} = 1.3$

$$\boxed{\text{Pearson} \rightarrow \frac{1.3}{\frac{\sqrt{14}}{2} \times \frac{\sqrt{70}}{10}} = \frac{13\sqrt{5}}{35} \simeq 0.8305}$$

Rank X_1	X_1	X_2	Rank X_2
1	-3	0	2.5
2	-2	0	2.5
3	-1	0	2.5
4	0	0	2.5
5	1	1	5
6	2	2	6

$$R_{x_2}(0) = \frac{1+2+3+4}{4} = 2.5$$

$$\overline{RX_1} = \frac{1+2+3+4+5+6}{6} = 3.5 \qquad \overline{RX_2} = \frac{2.5\times 4+5+6}{6} = 3.5$$

$X_{r_1 i}$	$X_{r_1 i} - \bar{X}_{r_1}$	$(X_{r_1 i} - \bar{X}_{r_1})^2$
1	-2.5	6.25
2	-1.5	2.25
3	-0.5	0.25
4	0.5	0.25
5	1.5	2.25
6	2.5	6.25

$$\rightarrow \sum_{i=1}^{n} (X_{r_1 i} - \bar{X}_{r_1})^2 = 17.5$$

$$\downarrow$$

$$S_{X_{r_1}} = \sqrt{\frac{17.5}{5}} = \frac{\sqrt{14}}{2}$$

$X_{r_2 i}$	$X_{r_2 i} - \bar{X}_{r_2}$	$(X_{r_2 i} - \bar{X}_{r_2})^2$
2.5	-1	1
2.5	-1	1
2.5	-1	1
2.5	-1	1
5	1.5	2.25
6	2.5	6.25

$$\rightarrow \sum_{i=1}^{n} (X_{r_2 i} - \bar{X}_{r_2})^2 = 12.5$$

$$\downarrow$$

$$S_{X_{r_2}} = \sqrt{\frac{12.5}{5}} = \frac{\sqrt{10}}{2}$$

$(X_{r_1 i} - \bar{X}_{r_1})(X_{r_2 i} - \bar{X}_{r_2})$
2.5
1.5
0.5
-0.5
2.25
6.25

$$\rightarrow \sum_{i=1}^{n} (X_{r_1 i} - \bar{X}_{r_1})(X_{r_2 i} - \bar{X}_{r_2}) = 12.5 \rightarrow Cov(X_{r_1}, X_{r_2}) = \frac{12.5}{5} = 2.5$$

$$\boxed{\text{Spearman} \rightarrow \frac{2.5}{\frac{\sqrt{14}}{2} \times \frac{\sqrt{10}}{2}} = \frac{\sqrt{35}}{7} \simeq 0.8452}$$

Ex1-c)

c) $x = (-3, -2, -1, 0, 1, 2)$ $\quad x_2 = f(x_1) = \frac{1}{1+e^{-x_1}}$

$$x_2 = \left(\frac{1}{e^3+1}, \frac{1}{e^2+1}, \frac{1}{e+1}, 0.5, \frac{1}{e^{-1}+1}, \frac{1}{e^{-2}+1} \right)$$

$S_{x_1} = \frac{\sqrt{14}}{2}$ ← calculated in 1a)

x_{2i}	$x_{2i} - \bar{x}_2$	$(x_{2i} - \bar{x}_2)^2$
$\frac{1}{e^3+1}$	-0.3771	0.1422
$\frac{1}{e^2+1}$	-0.3054	0.0932
$\frac{1}{e+1}$	-0.1556	0.0242
0.5	0.0754	0.0057
$\frac{1}{e^{-1}+1}$	0.3065	0.0939
$\frac{1}{e^{-2}+1}$	0.4562	0.2081

$\rightarrow \sum_{i=1}^{n} (x_{2i} - \bar{x}_2)^2 = 0.5673$

$\downarrow$

$S_{x_2} = \sqrt{\frac{0.5673}{5}} = 0.3368$

$$\bar{x}_2 = \frac{\frac{1}{e^3+1} + \frac{1}{e^2+1} + \frac{1}{e+1} + 0.5 + \frac{1}{e^{-1}+1} + \frac{1}{e^{-2}+1}}{6} \simeq 0.42457$$

$x_{1i} - \bar{x}_1$	$x_{2i} - \bar{x}_2$	$(x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)$
-2.5	-0.3771	0.94275
-1.5	-0.3054	0.4581
-0.5	-0.1556	0.0778
0.5	0.0754	0.0377
1.5	0.3065	0.45975
2.5	0.4562	1.1405

$\rightarrow \sum_{i=1}^{n} (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2) = 3.1165$

$\downarrow$

$Cov(x_1, x_2) = \frac{3.1165}{5} = 0.62332$

↑ calculated in 1a)

$$\boxed{\text{Pearson} \rightarrow \frac{0.62332}{0.3368 \times \frac{\sqrt{14}}{2}} \simeq 0.9841}$$

↳ almost perfect positive linear correlation

Rank X_1	X_1	X_2	Rank X_2
1	-3	$\frac{1}{e^3+1}$	1
2	-2	$\frac{1}{e^2+1}$	2
3	-1	$\frac{1}{e+1}$	3
4	0	0.5	4
5	1	$\frac{1}{e^{-1}+1}$	5
6	2	$\frac{1}{e^{-2}+1}$	6

Since Rank X_1 = Rank X_2, $\boxed{\text{Spearman} = 1}$

perfect positive monotonic correlation

Ex2-a)

Homework 1

Ex 2

Exam → E	Weather → W	Sunny → S	WhTodo! → WD
No → N	Sunny → Su	No → N	Go to the beach → B
Yes → Y	Rain → R	Yes → Y	Learn → L
	Cloudy → C		Reading → Read

a) $E(WD) = E(B, L, Read, Read, L, Read, B, L) =$

$= -p(WD=B)\log_2(p(WD=B)) - p(WD=L)\log_2(p(WD=L)) - p(WD=Read)\log_2(p(WD=Read))$

$= -\frac{2}{8}\log_2\left(\frac{2}{8}\right) - \frac{3}{8}\log_2\left(\frac{3}{8}\right) - \frac{3}{8}\log_2\left(\frac{3}{8}\right) = 1.561278$

$\underline{E} \rightarrow IG(WD|E) = E(WD) - E(WD|E) = 1.561278 - 0.606844$
$= 0.9544338785$

$E(WD|E) = \sum_{i=1}^{\hat{}} p(E=c_i)\, E(WD|E=c_i) =$

$= p(E=N)\,E(WD|E=N) + p(E=Y)\,E(WD|E=Y) =$

$= \frac{5}{8} \times E(B, Read, Read, Read, B) + \frac{3}{8} \times E(L, L, L) =$

$= \frac{5}{8} \times \left(-\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right)\right) + \frac{3}{8} \times 0 = 0.606844$

$\underline{\underline{W}}$ $IG(WD|W) = 0.8725561245$

$$E(WD|W) = \sum_{i=1}^{n} p(W=c_i)\, E(WD|W=c_i) =$$

$$= p(W=S)E(WD|W=S) + p(W=R)E(WD|W=R) + p(W=C)E(WD|W=C) =$$

$$= \frac{3}{8} E(B, L, B) + \frac{2}{8} E(Red, Red) + \frac{3}{8} E(Red, L, L) =$$

$$= \frac{3}{8} \times \left(-\frac{2}{3}\log_2\left(\frac{2}{3}\right) - \frac{1}{3}\log_2\left(\frac{1}{3}\right)\right) + \frac{2}{8} \times 0 + \frac{3}{8} \times \left(-\frac{1}{3}\log_2\left(\frac{1}{3}\right) - \frac{2}{3}\log_2\left(\frac{2}{3}\right)\right) =$$

$$= 2 \times \frac{3}{8}\left(-\frac{2}{3}\log_2\left(\frac{2}{3}\right) - \frac{1}{3}\log_2\left(\frac{1}{3}\right)\right) = 0.6887218755$$

$\underline{\underline{S}}$ $IG(WD|S) = 0.3600725407$

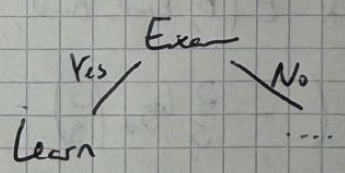
$$E(WD|S) = \sum_{i=1}^{n} p(S=c_i)\, E(WD|S=c_i) =$$

$$= p(S=Y)E(WD|S=Y) + p(S=N)E(WD|S=N) =$$

$$= \frac{5}{8} \times E(B, L, L, Red, L) + \frac{3}{8} E(Red, Red, B)$$

$$= \frac{5}{8} \times \left(-\frac{1}{5}\log_2\left(\frac{1}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right) - \frac{1}{5}\log_2\left(\frac{1}{5}\right)\right) + \frac{3}{8}\left(-\frac{2}{3}\log_2\left(\frac{2}{3}\right) - \frac{1}{3}\log_2\left(\frac{1}{3}\right)\right) =$$

$$= 1.201205051$$



$IG(WD|E) = 0.95$

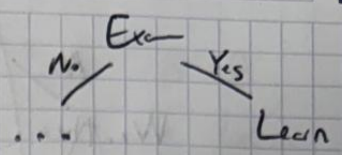
$IG(WD|W) = 0.87$

$IG(WD|S) = 0.36$

Ex2-b)

b)

Exam	Weather	Summer	What to do?
No	Sunny	Yes	Go to the beach
No	Rain	No	Reading
No	Cloudy	No	Reading
No	Rain	Yes	Reading
No	Sunny	No	Go to the beach



$E(WD) = E(B, Read, Read, Read, B) =$

$= -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right) = 0.970950594$

$\underline{\underline{W}}$ $IG(WD|W) = 0.970950594$

$E(WD|W) = \sum_{i=1}^{n} p(W=c_i)\, E(WD|W=c_i) =$

$= p(W=Su)\, E(B,B) + p(W=R)\, E(Read, Read) + p(W=C)\, E(Read) =$

$= \frac{2}{5} \times \left(-1\log(1)\right) + \frac{2}{5} \times \left(-1\log(1)\right) + \frac{1}{5}\left(-1\log(1)\right) = 0$

$\underline{\underline{S}}$ $IG(WD|S) = 0.019973094 07$

$E(WD|S) = \sum_{i=1}^{n} p(S=c_i)\, E(WD|S=c_i) =$

$= p(S=Y)\, E(WD|S=Y) + p(S=N)\, E(WD|S=N) =$

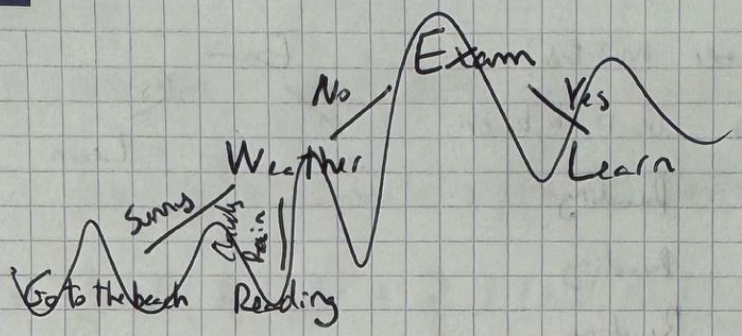
$= \frac{2}{5} \times E(B, Read) + \frac{3}{5} \times E(Read, Read, B) =$

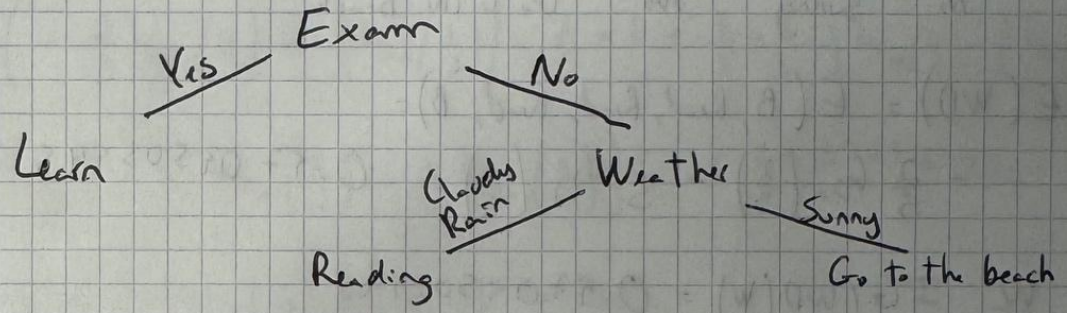
$= \frac{2}{5} \times 2 \times \left(-\frac{1}{2}\log_2\left(\frac{1}{2}\right)\right) + \frac{3}{5} \times \left(-\frac{2}{3}\log_2\left(\frac{2}{3}\right) - \frac{1}{3}\log_2\left(\frac{1}{3}\right)\right) =$

$= 0.950977500 4$

$IG(WD|S) = 0.02$

$IG(WD|W) = 0.97$


Exam
No
Yes
Weather
Learn
Sunny
Clouds Rain
Go to the beach
Reading


Exam
Yes
No
Learn
Weather
Clouds Rain
Sunny
Reading
Go to the beach

Ex2-c)

c)

Real:	B	Learn	Read	Read	Learn	Read	Learn	Learn
Prediction:	~~B~~	~~B~~	Read	~~B~~	~~B~~	Learn	~~B~~	~~B~~

		True		
		B	Read	Learn
	B	1	1	4
Predicted	Read	0	1	0
	Learn	0	1	0

Ex3- a)

The question would be as follows:

“Please generate a self-contained Python script that implements the ID3 algorithm from scratch and applies it to the dataset below (target = RISK). Requirements: pure Python ID3 implementation (no sklearn decision trees), use entropy/information gain, handle categorical values (treat unknown as a category), print the final tree (nested dict), print training accuracy and per-row predictions, and produce a hierarchical matplotlib visualization saved as id3_tree.png. Organize the code into clear functions with docstrings and comments. Use pandas/numpy only if desired. Provide no extra explanatory text outside of the code.

NO.,RISK,CREDIT HISTORY,DEBT,COLLATERAL,INCOME
1,high,bad,high,none,$0 to $15k
2,high,unknown,high,none,$15 to $35k
3,moderate,unknown,low,none,$15 to $35k
4,high,unknown,low,none,$0 to $15k
5,low,unknown,low,none,over $35k
6,low,unknown,low,adequate,over $35k
7,high,bad,low,none,$0 to $15k
8,moderate,bad,low,adequate,over $35k
9,low,good,low,none,over $35k
10,low,good,high,adequate,over $35k
11,high,good,high,none,$0 to $15k
12,moderate,good,high,none,$15 to $35k
13,low,good,high,none,over $35k
14,high,bad,high,none,$15 to $35k”

We used that prompt with ChatGPT-5, and got the following code:

```python
# -*- coding: utf-8 -*-
"""
Self-contained ID3 implementation and demo.
Saves a hierarchical matplotlib visualization as id3_tree.png

Usage: run this script with Python 3.6+ (pandas and matplotlib required).
This file prints:
 - the final tree as a nested dict (JSON)
 - a pretty printed hierarchical tree (nested text)
 - per-row predictions and correctness
 - training accuracy
 - saves id3_tree.png in the working directory
"""

import math
from collections import Counter
import matplotlib.pyplot as plt
import pandas as pd

# --- Dataset (embedded) ---
CSV = """NO.,RISK,CREDIT HISTORY,DEBT,COLLATERAL,INCOME
1,high,bad,high,none,$0 to $15k
2,high,unknown,high,none,$15 to $35k
3,moderate,unknown,low,none,$15 to $35k
4,high,unknown,low,none,$0 to $15k
5,low,unknown,low,none,over $35k
6,low,unknown,low,adequate,over $35k
7,high,bad,low,none,$0 to $15k
8,moderate,bad,low,adequate,over $35k
9,low,good,low,none,over $35k
10,low,good,high,adequate,over $35k
11,high,good,high,none,$0 to $15k
12,moderate,good,high,none,$15 to $35k
13,low,good,high,none,over $35k
14,high,bad,high,none,$15 to $35k
"""

# --- Data loading ---
def load_data_from_csv_string(csv_string):
    """
    Load CSV data from a string into a pandas DataFrame.
    Drops the NO. column if present.
    """
    from io import StringIO
    df = pd.read_csv(StringIO(csv_string))
    if 'NO.' in df.columns:
        df = df.drop(columns=['NO.'])
    return df
```

```python
# --- ID3 core utilities ---
def entropy(rows, target_col):
    """
    Compute entropy of the target column for the provided rows DataFrame.
    """
    counts = rows[target_col].value_counts().to_dict()
    total = len(rows)
    if total == 0:
        return 0.0
    ent = 0.0
    for v in counts.values():
        p = v / total
        if p > 0:
            ent -= p * math.log2(p)
    return ent

def information_gain(rows, attr, target_col):
    """
    Compute information gain from splitting rows on attr.
    """
    base_entropy = entropy(rows, target_col)
    total = len(rows)
    remainder = 0.0
    # group by categorical values
    for _, group in rows.groupby(attr):
        weight = len(group) / total
        remainder += weight * entropy(group, target_col)
    return base_entropy - remainder

def majority_class(rows, target_col):
    """
    Return the most common label in target_col for rows.
    If tie, pandas.mode() returns smallest by order; that's acceptable here.
    """
    if len(rows) == 0:
        return None
    return rows[target_col].mode().iloc[0]
```

```python
def id3(rows, attributes, target_col, depth=0, max_depth=None):
    """
    ID3 recursive algorithm.
    Returns a nested dict:
    - Leaf: {'leaf': label}
    - Internal: {'attribute': attr, 'nodes': {value: subtree, ...}}
    """
    # If all examples have same target -> leaf
    unique_targets = rows[target_col].unique()
    if len(unique_targets) == 1:
        return {'leaf': unique_targets[0]}
    # No attributes left or no rows -> majority class
    if not attributes or len(rows) == 0:
        return {'leaf': majority_class(rows, target_col)}
    # Max depth guard (optional)
    if max_depth is not None and depth >= max_depth:
        return {'leaf': majority_class(rows, target_col)}
    # Choose best attribute by information gain
    gains = {attr: information_gain(rows, attr, target_col) for attr in attributes}
    best_attr = max(gains, key=gains.get)
    # If best gain is zero, return leaf with majority class (can't improve)
    if gains[best_attr] <= 0:
        return {'leaf': majority_class(rows, target_col)}
    tree = {'attribute': best_attr, 'nodes': {}}
    # For every value of best_attr create subtree
    for val in rows[best_attr].unique():
        subset = rows[rows[best_attr] == val].copy()
        new_attrs = [a for a in attributes if a != best_attr]
        subtree = id3(subset, new_attrs, target_col, depth + 1, max_depth)
        tree['nodes'][val] = subtree
    return tree
```

```python
# --- Prediction helpers ---
def predict_row(tree, row):
    """
    Predict single row (dict-like) using the tree.
    If unseen attribute value encountered, fallback strategies:
      1) if 'unknown' branch exists, follow it
      2) else return majority leaf among children
    """
    node = tree
    while True:
        if 'leaf' in node:
            return node['leaf']
        attr = node['attribute']
        val = row.get(attr)
        if val in node['nodes']:
            node = node['nodes'][val]
        else:
            # fallback
            if 'unknown' in node['nodes']:
                node = node['nodes']['unknown']
                continue
            # collect leaves and return majority
            child_leaves = []
            def collect_leaves(n):
                if 'leaf' in n:
                    child_leaves.append(n['leaf'])
                else:
                    for c in n['nodes'].values():
                        collect_leaves(c)
            collect_leaves(node)
            if child_leaves:
                return Counter(child_leaves).most_common(1)[0][0]
            return None

def predict_dataframe(tree, df):
    """
    Predict for each row in DataFrame df.
    Returns list of predictions.
    """
    preds = []
    for _, row in df.iterrows():
        preds.append(predict_row(tree, row.to_dict()))
    return preds
```

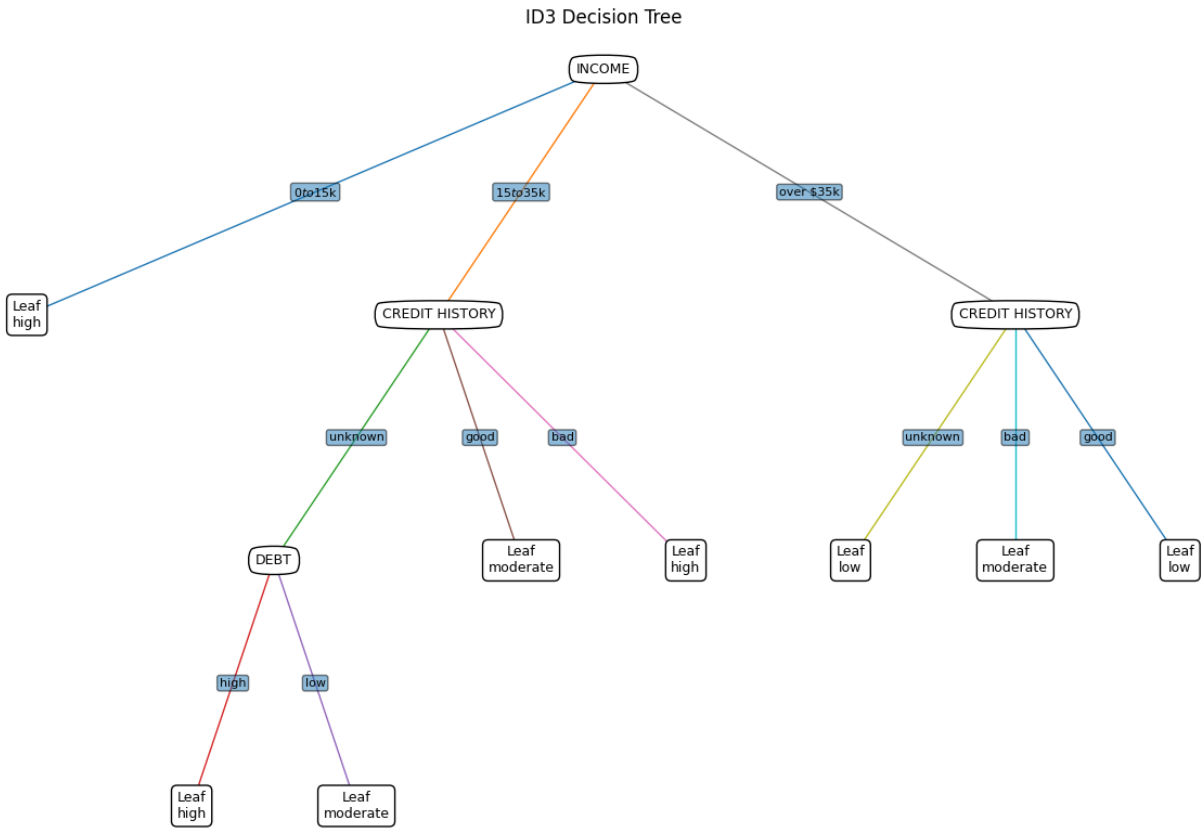
```python
# --- Pretty print tree ---
def print_tree(tree, indent=0):
    """
    Nicely print the nested dict tree structure for human reading.
    """
    spacer = "  " * indent
    if 'leaf' in tree:
        print(f"{spacer}Leaf: {tree['leaf']}")
        return
    attr = tree['attribute']
    print(f"{spacer}{attr}?")
    for val, subtree in tree['nodes'].items():
        print(f"{spacer} -> {val}:")
        print_tree(subtree, indent + 2)

# --- Visualization (matplotlib) ---
def count_leaves(tree):
    """
    Count leaves under a tree node.
    """
    if 'leaf' in tree:
        return 1
    return sum(count_leaves(c) for c in tree['nodes'].values())
```

```python
def layout_positions(tree, x_min=0.0, x_max=1.0, y_spacing=0.12, level=0, positions=None, labels=None):
    """
    Assign (x,y) positions to every node/leaf for plotting.
    Positions keyed by id(node).
    """
    if positions is None:
        positions = {}
    if labels is None:
        labels = {}
    node_id = id(tree)
    if 'leaf' in tree:
        xpos = (x_min + x_max) / 2.0
        positions[node_id] = (xpos, 1.0 - level * y_spacing)
        labels[node_id] = f"Leaf\n{tree['leaf']}"
        return positions, labels
    total = count_leaves(tree)
    cur = x_min
    for val, child in tree['nodes'].items():
        cl = count_leaves(child)
        span = (cl / total) * (x_max - x_min) if total > 0 else 0
        child_min = cur
        child_max = cur + span
        layout_positions(child, child_min, child_max, y_spacing, level + 1, positions, labels)
        cur += span
    xpos = (x_min + x_max) / 2.0
    positions[node_id] = (xpos, 1.0 - level * y_spacing)
    labels[node_id] = f"{tree['attribute']}"
    return positions, labels
```

```python
def draw_tree(tree, filename="id3_tree.png"):
    """
    Draw the tree and save to filename (PNG).
    """
    positions, labels = layout_positions(tree, x_min=0.02, x_max=0.98, y_spacing=0.14)
    fig, ax = plt.subplots(figsize=(12, 8))
    ax.set_axis_off()
    # draw edges recursively
    def draw_edges(node):
        nid = id(node)
        if 'leaf' in node:
            return
        px, py = positions[nid]
        for val, child in node['nodes'].items():
            cid = id(child)
            cx, cy = positions[cid]
            ax.plot([px, cx], [py, cy], linewidth=1)
            # edge label at midpoint
            mx, my = (px + cx) / 2.0, (py + cy) / 2.0
            ax.text(mx, my, str(val), fontsize=8, ha='center', va='center', bbox=dict(boxstyle="round,pad=0.2",
alpha=0.5))
            draw_edges(child)
    draw_edges(tree)
    # draw nodes
    for nid, (xpos, ypos) in positions.items():
        label = labels[nid]
        if label.startswith("Leaf"):
            bbox = dict(boxstyle="round,pad=0.4", edgecolor="black", facecolor="white")
        else:
            bbox = dict(boxstyle="round4,pad=0.6", edgecolor="black", facecolor="white")
        ax.text(xpos, ypos, label, fontsize=9, ha='center', va='center', bbox=bbox)
    ax.set_title("ID3 Decision Tree")
    plt.tight_layout()
    fig.savefig(filename)
    plt.close(fig)
    return filename
```

```python
# --- Main execution ---
if __name__ == "__main__":
    df = load_data_from_csv_string(CSV)
    target = "RISK"
    attributes = [c for c in df.columns if c != target]
    # Train ID3 tree
    tree = id3(df, attributes, target)
    # Print the nested dict (JSON-like)
    import json
    def tree_to_jsonable(t):
        if 'leaf' in t:
            return {'leaf': t['leaf']}
        return {'attribute': t['attribute'], 'nodes': {k: tree_to_jsonable(v) for k, v in t['nodes'].items()}}
    print(json.dumps(tree_to_jsonable(tree), indent=2))
    # Pretty print
    print("\nPretty tree:")
    print_tree(tree)
    # Per-row predictions and accuracy
    preds = predict_dataframe(tree, df)
    df_results = df.copy()
    df_results['_pred'] = preds
    df_results['_correct'] = df_results[target] == df_results['_pred']
    accuracy = df_results['_correct'].mean()
    print("\nPer-row predictions:")
    print(df_results.to_string(index=False))
    print(f"\nTraining accuracy: {accuracy*100:.2f}%")
    # Save visualization
    out = draw_tree(tree, filename="id3_tree.png")
    print(f"\nTree visualization saved to: {out}")
```

The code generated the following tree:



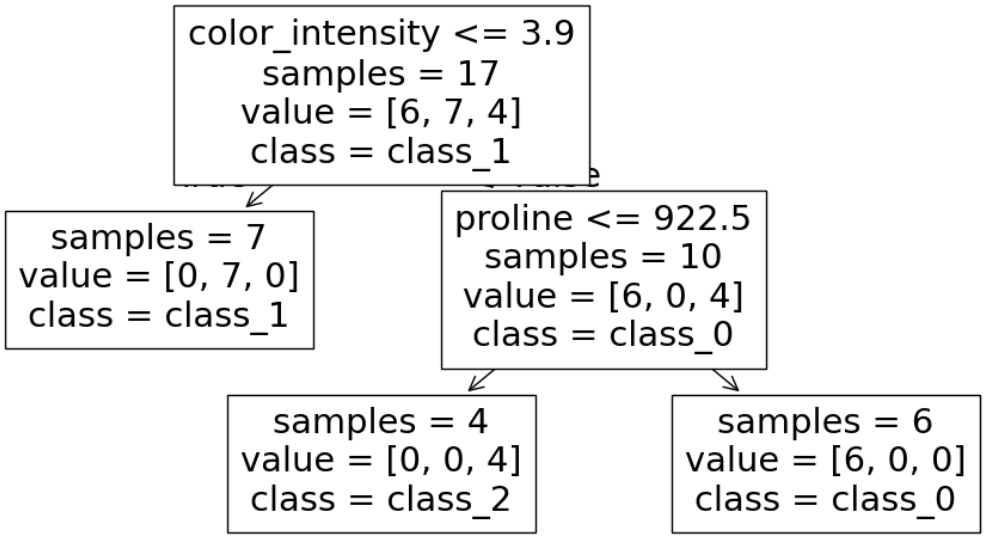
A json-like tree was also generated, showing a similar output.

The generated code produced the expected results and was overall correct. Nevertheless, the solution appeared overly complex, with unnecessary steps that a human programmer would usually avoid.

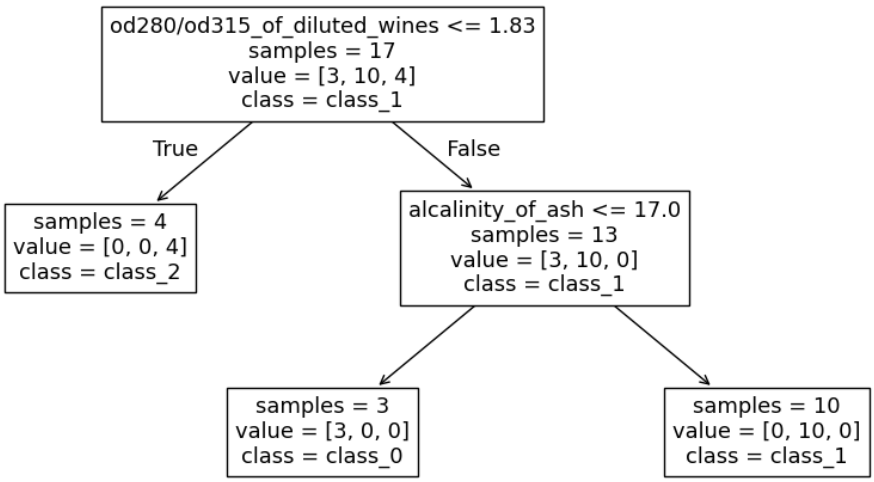
We also tested with alternative prompts, but these were less effective and often produced incorrect answers.

Ex3 -b)

ID3 graphical visualization **with** stratify:



ID3 graphical visualization **without** stratify:



With stratify=y, the class distribution in the training set is preserved, leading to more balanced splits, but without stratification, the training set becomes unbalanced, so the decision tree structure changes accordingly. That's why with stratify the initial values are [6,7,4] and without stratify they are [3,10,4].

Ex3- c)

Without stratification, the training set does not preserve the original class distribution, leading to a biased model that generalizes worse and thus lowers accuracy.